Applied Statistical Programming

Week 1

Logistics

Course meetings

- Lectures: Tuesdays, 10:00 am–12:00 pm
- Location: Seigle 207
- Labs: Fridays, 9:00–9:50 am
- Lab location: TBD

Instructor

Ted Enamorado
Department of Political Science
Office: Seigle 244
Email: `ted@wustl.edu`
Office hours: Tuesdays, 2:00–4:00 pm or by appointment

Assistant Instructor

Xiangyu Song
Office: Seigle 257
Email: `xiangyusong@wustl.edu`
Office hours: TBD

Course requirements

Grade components

- Labs: 10 points
  - Attendance: 5 points
  - Lab exercises: 5 points
- Problem sets: 60 points
- Final exam: 30 points

Expectations

- Attend all weekly lab sessions.
- Problem sets combine analytical and data-analysis questions.
- Five problem sets will contribute equally to the final grade.

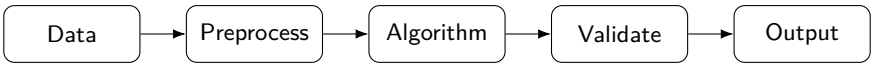
Tentative course plan

1. Computational workflow
2. Efficient R
3. Resampling: bootstrap and permutation tests
4. Parallel computing and scalable resampling
5. Clustering: K-means and Gaussian mixture models
6. EM algorithm: mixture of multinomials for text
7. Pairwise comparisons: cosine similarity, LSH, approximate search
8. Rcpp and optimized pairwise algorithms
9. Simulation for text-analysis methods
10. Ideal-point estimation: Bayesian approach
11. Ideal-point estimation: variational inference
12. Embeddings
13. Active learning
14. Integration, package development, and project presentations

AI policy

- AI tools are allowed and encouraged for learning, debugging, documentation, and improving code.
- AI should function as a personalized code reviewer, not a replacement for your own implementation.
- You may not paste complete AI-generated functions into submissions, submit code you do not understand, or ask AI to complete an entire assignment.
- You are responsible for every line of submitted code and may be asked to explain your implementation choices.
- AI assistance must be disclosed in a short commented note at the top of the main R script.

Research is a pipeline



- A reproducible workflow makes every connection explicit: we should be able to identify exactly how the data are transformed at each step and reproduce those transformations from the code.

- **Software:** each arrow should have well-defined inputs, outputs, and expected behavior so that individual components can be tested and modified without breaking the entire pipeline.

- **Methods:** each arrow may have consequences for the analysis. We therefore ask whether a transformation changes the estimand or affects how uncertainty should be quantified.

Project structure:

Separate inputs, code, and outputs

```
project/
  R/           # reusable functions used across analyses or scripts
  data-raw/    # original input data; keep these files unchanged
  data/        # cleaned or transformed data created from data-raw/
  analysis/    # scripts or notebooks that produce substantive analyses
  tests/       # automated checks that verify functions behave as expected
  output/      # generated tables, figures, and other analysis results
  README.md    # instructions describing the project and how to reproduce it
```

- Never encode your machine's absolute path in analysis code.
- Generated outputs should be regenerable from code.
- Functions should migrate out of analysis scripts once they become reusable.

Relative paths

- A path is part of the program's interface with the filesystem.
- Portability requires paths to be defined relative to the project root.
- Reproducibility begins before the statistical model: it starts with file organization.

```
# Bad: only works on one machine
x <- readRDS('/Users/name/Desktop/project/data/x.rds')

# Better: relative to the current project directory
x <- readRDS('./data/x.rds')

# Relative paths also work for nested folders
df <- read.csv('./data/raw/survey.csv')

# And for writing output
saveRDS(df, './output/clean_data.rds')

# Equivalent using file.path()
x <- readRDS(file.path('.', 'data', 'x.rds'))

# Convenient when using the here package
x <- readRDS(here::here('data', 'x.rds'))
```

Functions

A function can be viewed mathematically as a map

$$f : \mathcal{X} \to \mathcal{Y}, \qquad y = f(x; \theta).$$

A useful software contract specifies:

- **Domain**: allowed types, dimensions, missingness, parameter ranges.
- **Transformation**: what statistical/computational operation is performed.
- **Codomain**: return type, dimensions, names, side effects.
- **Failure behavior**: informative error and warnings.

Structure

```
# Define a function with one required input, x (to be rescaled) and two optional arguments controlling centering
      and scaling
center_scale <- function(x, center = TRUE, scale = TRUE) {
  # Stop if x is not numeric or has more than two dimensions
  stopifnot(is.numeric(x), length(dim(x)) <= 2L)

  # Stop with an informative error if x contains missing values
  if (anyNA(x)) stop('x contains missing values')

  # Center and/or scale x using base R's scale() function
  out <- base::scale(x, center = center, scale = scale)

  # Remove attributes added by scale() so the output structure is simpler
  attr(out, 'scaled:center') <- NULL

  # Remove the stored scaling values for the same reason
  attr(out, 'scaled:scale')  <- NULL

  # Return the transformed object
  out

}
```

- Input checks: turn issues into visible failures.
- A stable return structure lets downstream code depend on the function safely.

Pure (deterministic) functions

A **pure** function has two useful properties:

$f(x) = f(x)$ every time, and it does not change anything outside the function.

```
# Not pure: changes x outside the function
x <- 1

f <- function() {
  x <<- x + 1
}

# Pure: takes x as input and returns a result
f <- function(x) {
  x + 1
}
```

- Pure functions are easier to test, run in parallel, and understand.
- Randomized algorithms can get close to this ideal by making the random seed part of the computation.
- Side effects, such as writing files or changing options, should happen outside the function.

Reproducible randomness is an input

For a simulation estimator

$$\widehat{\theta}_M = \frac{1}{M}\sum_{m=1}^{M} g(U_m), \qquad U_m \sim F_U,$$

the random part determines the realized $\widehat{\theta}_M$.

```
sim_once <- function(seed, n = 100) {
  set.seed(seed)
  x <- rnorm(n)
  c(mean = mean(x), sd = sd(x))
}

sim_once(2026)
sim_once(2026)  # identical
```

- Record seeds for debugging and for exact replication.

Documentation

```r
#' Cosine similarity
#'
#' @param x,y Numeric vectors of equal length.
#' @return Scalar in [-1,1], or NA if a norm is zero.
#' @export
cosine_similarity <- function(x, y) {
  stopifnot(length(x) == length(y))
  nx <- sqrt(sum(x^2)); ny <- sqrt(sum(y^2))
  if (nx == 0 || ny == 0) return(NA_real_)
  sum(x * y) / (nx * ny)
}
```

- The documentation states the mathematical object and the edge-case convention.

Tests

If a function says $s(x, x) = 1$ for every nonzero vector x, write that as a test.

```
testthat::test_that('cosine similarity satisfies basic identities', {
  x <- c(1, -2, 3)
  y <- c(4,  1, 0)
  testthat::expect_equal(cosine_similarity(x, x), 1)
  testthat::expect_equal(cosine_similarity(x, y),
                         cosine_similarity(y, x))
  testthat::expect_true(abs(cosine_similarity(x, y)) <= 1)
})
```

Benchmarking

Let $T_A(x)$ be runtime of implementation A on input x. A speed claim should specify

$$\text{speedup}(x) = \frac{T_{\text{baseline}}(x)}{T_{\text{new}}(x)}.$$

- Benchmark across representative input sizes, not only one toy case.
- Separate compilation/setup costs from repeated execution costs.
- Verify that the faster implementation returns statistically equivalent results before celebrating speed.

A reproducible benchmark

```
bench_one <- function(n, reps = 20L) {
  x <- rnorm(n); y <- rnorm(n)
  t1 <- replicate(reps, system.time(cosine_similarity(x,y))['elapsed'])
  t2 <- replicate(reps, system.time(sum(x*y)/sqrt(sum(x*x)*sum(y*y)))['elapsed'])
  data.frame(n = n,
             method = c('function','inline'),
             median = c(median(t1), median(t2)))
}
```

- Later we will replace this minimal example with better profiling/benchmark tools, but the logic stays the same.

Code review asks statistical questions too

A review should ask more than "does this code run?"

1. Does the code implement the stated estimand/algorithm?
2. Are edge cases handled consistently with the assumptions?
3. Is randomness controlled and documented?
4. Are numerical approximations stable?
5. Are performance claims benchmarked on representative inputs?

Version control

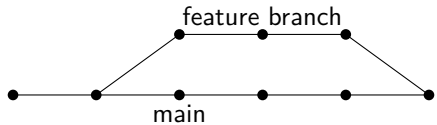
Version control records transformations of the project. Think of a commit as a state transition

$$S_t \xrightarrow{\Delta_t} S_{t+1},$$

where S_t is the repository state and Δ_t is a documented set of changes.

- Commits make failures discoverable: which transformation introduced the bug?
- Branches isolate experimental algorithms from a stable baseline.

Branches are experiments under controlled integration



- A **branch** creates an isolated version of the project: you can change code without immediately changing the stable version used by collaborators or analyses.
- This is useful when the proposed change is **uncertain or experimental**. For example, we might implement a new estimator or optimization procedure without knowing whether it will improve the existing method.
- The branch can be compared against `main`: do the old tests still pass? Do simulations improve? Are results unchanged in cases where they should be unchanged?
- Only after those checks do we **merge** the branch, incorporating the successful experiment into the main project.

Git and GitHub: the big picture

- **Git** is a version-control system: it records changes to your project over time, similar to "Track Changes," but for an entire collection of files.
- **GitHub** is an online platform for storing and collaborating on Git repositories. It provides a remote copy of your project that can be shared with others.
- A typical project therefore has two versions:
  - **Local**: the copy on your computer.
  - **Remote**: the copy hosted on GitHub.
- Git has many commands, but most day-to-day work relies on only a few:

$$\texttt{add} \rightarrow \texttt{commit} \rightarrow \texttt{push}, \qquad \texttt{pull}$$

Repositories and synchronization

- **Repository (repo):** the project folder tracked by Git, including its files and revision history.
- **Clone:** create a local copy of an existing repository.

  `GitHub repository` $\longrightarrow$ `your computer`
- **Remote:** the online version of the repository, usually hosted on GitHub.
- **Pull:** bring changes from the remote repository into your local copy.

  `GitHub` $\longrightarrow$ `local`
- **Push:** send your committed local changes to the remote repository.

  `local` $\longrightarrow$ `GitHub`

Commits and collaboration on GitHub

- **Commit:** a saved checkpoint in the project's history. A commit records a specific set of changes together with a message describing what changed.

```
git commit -m "Fix missing-value handling"
```

- **Fork:** your own GitHub copy of someone else's repository. Useful when you want to modify a project but do not have permission to change the original directly.
- **Pull request:** a proposal to merge your changes into another branch or repository. Collaborators can review, discuss, accept, or reject the changes.
- **Issue:** a place to record bugs, questions, feature requests, or tasks associated with the project.

When should I commit?

- **Commit early and often.** Think of commits as checkpoints in a video game: they give you safe points to return to.
- A good commit represents **one meaningful change**, such as:
  - fixing a bug,
  - adding a function,
  - adding a test,
  - changing a statistical specification,
  - updating documentation.
- Write messages that explain **what changed**, rather than vague messages such as `"updates"` or `"changes"`.
- Do not wait until the end of the day to create one enormous commit. Smaller commits make it easier to identify when and why something changed.

Example: developing a new feature in `fastLink`

Suppose we want to experiment with a new way of measuring name similarity.

```
# Stable version of the package
git switch main

# Create an isolated branch for the experiment
git switch -c feature/name-similarity
```

- **Current:** `fastLink` uses e.g., Jaro–Winkler similarity for string fields such as names.
- **Branch:** implement an alternative similarity measure (e.g., cosine) without changing `main`.
- **Validate:** compare linkage accuracy, match probabilities, and computational cost.
- **Decision:** merge if it improves performance; otherwise discard the branch.

From script to package

A package is a disciplined way to separate **public interface** from **implementation details**.

```
mypkg/
  DESCRIPTION       # package name, version, dependencies, authors
  NAMESPACE         # functions made available to users
  R/                # R functions and internal helper code
  man/              # documentation for functions and datasets
  tests/testthat/   # automated tests for expected behavior
  vignettes/        # longer tutorials and worked examples
  src/              # compiled code, e.g., C or C++
```

- Users should call a small number of stable exported functions (functions users are expected to call and rely on).
- Internal helper functions can change without breaking user code.
- Documentation, tests, and compiled code live beside the implementation.

Create a package skeleton early

```
# Create a new R package called textMethods
usethis::create_package('textMethods')

# Initialize Git version control for the package
usethis::use_git()

# Set up the testthat framework for automated tests
usethis::use_testthat()

# Configure roxygen2 for function documentation
usethis::use_roxygen_md()

# Create R/cosine_similarity.R for the function
usethis::use_r('cosine_similarity')

# Create a corresponding test file in tests/testthat/
usethis::use_test('cosine_similarity')
```

- We will add methods to the skeleton throughout the semester.
- Package structure is not the end; it shapes how we decompose problems from the beginning.

This week's Lab: make a sampling script reproducible

You receive a script that draws a random sample from a population and estimates the sample mean. The script uses global objects, hard-coded paths, and an unseeded sample.

1. Write a function that draws a simple random sample, with the population, sample size, and seed as explicit inputs.
2. Write a function that computes the sample mean from the selected units.
3. Replace hard-coded input and output paths with project-relative paths.
4. Write two tests:
   - **"Identity":** if the sample is drawn using the same seed, it is always equal.
   - **Impossible case:** return an informative error if the requested sample size is larger than the population.
5. Commit the new analysis on a branch.